

Notes Week 17

a. Semi-analytic Boundaries for Non-Conservative Mass Loss

Once the radius of a star exceeds the semi-major axis of the binary, the system is in contact. This is an critical boundary for “normal” Roche Lobe overflowing systems.

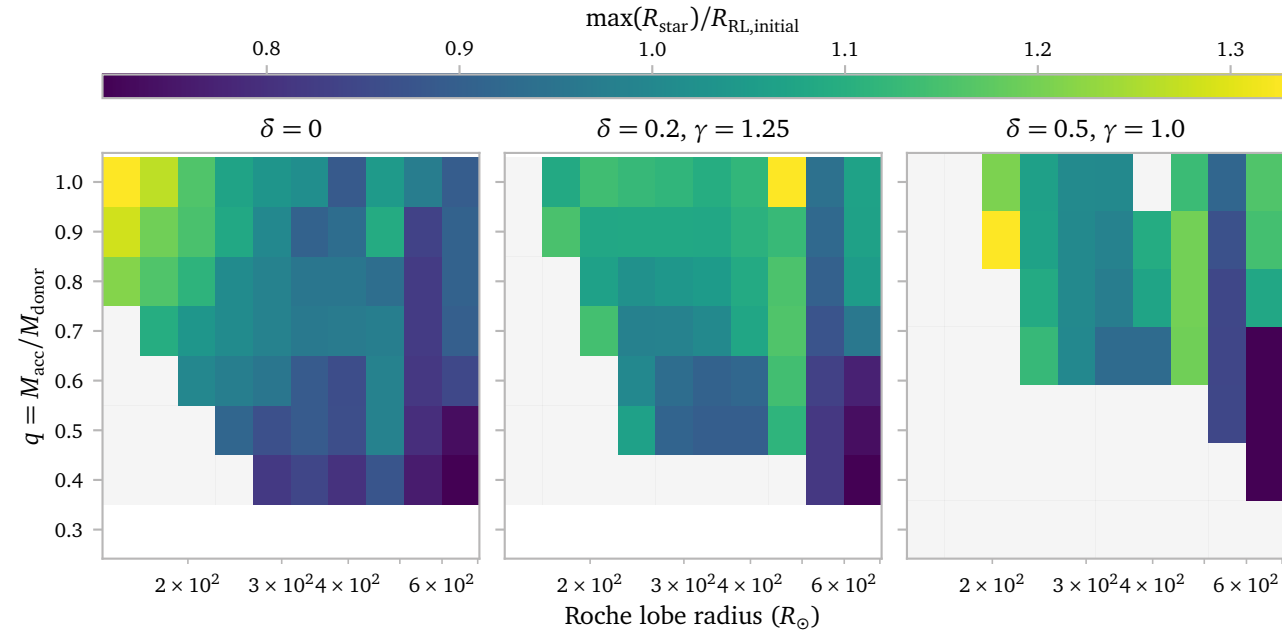
a.1 Implementation

We can use the analytical expressions from Soberman et al. (1997) to compute the minimum semi-major axis that is reached during a mass-transfer episode. If we combine this with a model for the maximum radius that a star reaches during mass-transfer, we can find critical efficiency parameters for mass loss.

Unfortunately, the radial response of realistic star cannot accurately be captured by an analytical expression, so we have to use some approximations.

My idea is to use the radius of the mass-transferring models that I simulated with MESA. I use these radii to fit a polynomial and obtain an expression $R_{\max}(q, R_{\text{RL}})$.

Below, I plot the maximum radius that the models reach divided by their starting Roche lobe radius as a form of normalization.



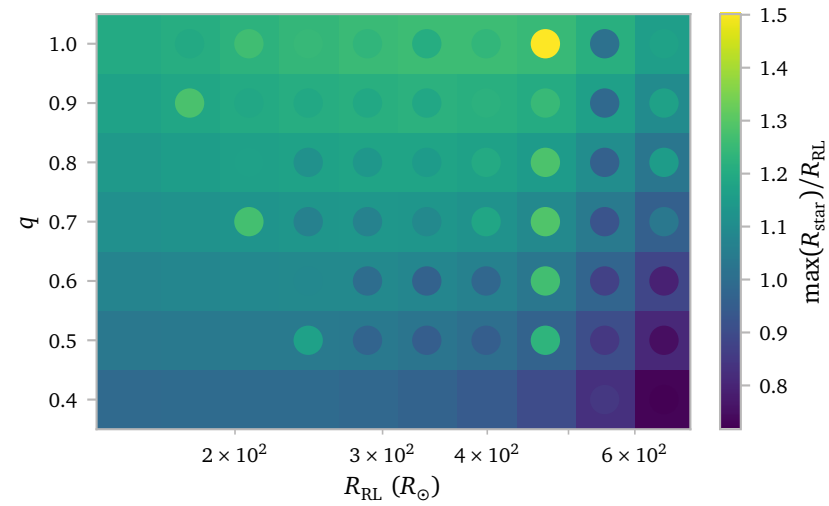
Unfortunately, but not entirely unexpectedly, the choice of mass transfer efficiency changes the stellar response to mass loss¹. For now, I continue only with the results from the $\delta = 0.2, \gamma = 1.25$ models but I should definitely keep in mind that this introduces significant uncertainties.

I think it makes most sense to fit a simple 2d polynomial polynomial. By looking at the overall trends in the star radius, I think a second degree polynomial should be able to capture the general behaviour quite well. I use:

$$\frac{R_{\max}(q, R_{\text{RL}})}{R_{\text{RL}}} = aR_{\text{RL}}^2 + bq^2 + cR_{\text{RL}}q + dR_{\text{RL}} + eq + f.$$

Then, I simply use `scipy.optimize.curve_fit` to fit the data to the polynomial.

[Here](#), I have an interactive view of the fit to the data points. Below, I plot a static view.

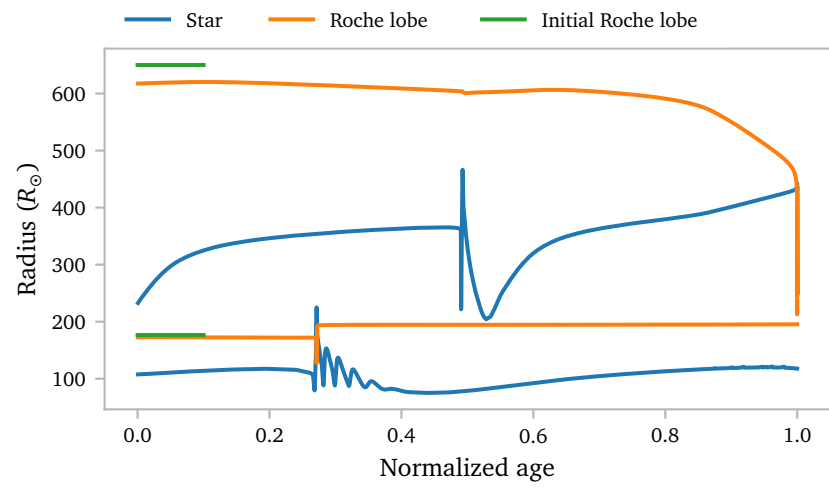


¹ This is because different mass transfer efficiencies change the size of the orbit quite drastically. This in turn changes the mass transfer rate, which seems to impact the radial response of the star.

The obvious advantage of the fit is that it is now trivially easy to compute the radius of the star for any given initial model condition. Even though I still plot the values over a discrete interval, this is by no means necessary.

I think it is somewhat interesting to see why for lower starting Roche lobe radii, the star seems to reach relatively larger final radii. Let me plot some models.

Here, I compare a $R_{\text{RL},i} = 176.54 R_{\odot}, q = 0.9$ model with a $R_{\text{RL},i} = 650 R_{\odot}, q = 0.4$ model. The efficient shrinking of the orbit due to wind mass transfer causes the ratio to decrease.



Next, we need to find the minimum orbital separation reached due to mass transfer according to the prescriptions by Soberman et al. (1997). This is actually pretty easy, since the expression for a/a_0 is simple too.

We have:

$$\frac{a}{a_0} = \left(\frac{q}{q_0}\right)^{2\mathscr{A}-2} \left(\frac{1+q}{1+q_0}\right)^{1-2\mathscr{B}} \left(\frac{1+\varepsilon q}{1+\varepsilon q_0}\right)^{2\mathscr{C}+3}.$$

This is easily differentiated using logarithmic differentiation.

$$\begin{aligned} \left(\frac{a}{a_0}\right)' &= \left(\frac{a}{a_0}\right) \cdot \left[\ln\left(\frac{a}{a_0}\right)\right]' \\ \ln\left(\frac{a}{a_0}\right) &= (2\mathscr{A}-2)\ln\left(\frac{q}{q_0}\right) + (1-2\mathscr{B})\ln\left(\frac{1+q}{1+q_0}\right) \\ &\quad + (2\mathscr{C}+3)\ln\left(\frac{1+\varepsilon q}{1+\varepsilon q_0}\right) \\ \frac{da}{dq} &= a \left(\frac{2\mathscr{A}-2}{q} + \frac{1-2\mathscr{B}}{1+q} + \frac{\varepsilon(2\mathscr{C}+3)}{1+\varepsilon q} \right) \end{aligned}$$

Now we can use this equation to find stationary points.

We must have

$$\frac{1}{a} \frac{da}{dq} = \frac{2\mathscr{A}-2}{q} + \frac{1-2\mathscr{B}}{1+q} + \frac{\varepsilon(2\mathscr{C}+3)}{1+\varepsilon q} = 0.$$

let $n_1 = 2\mathscr{A}-2$, $n_2 = 1-2\mathscr{B}$ and $n_3 = 2\mathscr{C}+3$

$$\begin{aligned} 0 &= n_1(1+q)(1+\varepsilon q) + n_2(q)(1+\varepsilon q) + \varepsilon n_3(q)(1+q) \\ 0 &= n_1(1+(1+\varepsilon)q + \varepsilon q^2) + n_2q + \varepsilon n_2q^2 + \varepsilon n_3q + \varepsilon n_3q^2. \end{aligned}$$

Now we can apply the quadratic formula using

$$\begin{aligned} a &= \varepsilon(n_1 + n_2 + n_3) \\ b &= n_1(1+\varepsilon) + n_2 + \varepsilon n_3 \\ c &= n_1. \end{aligned}$$

I select only the positive roots, and I make sure q is inside possible range of q values for this star.

Then, in order to make sure the stationary point is a minimum, we use the second derivative. We have

$$\frac{d^2a}{dq^2} = \frac{da}{dq} \cdot \frac{1}{a} \frac{da}{dq} + a \cdot \frac{d}{dq} \left[\frac{1}{a} \frac{da}{dq} \right].$$

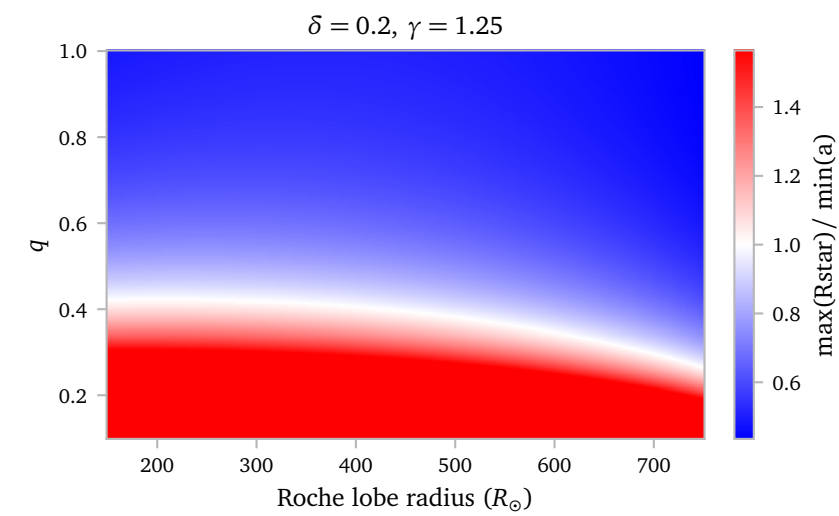
Since da/dq at a stationary point is equal to zero, and since $a > 0$, we only need to look at the sign of $d/dq [1/a \cdot da/dq]$. If it is positive, we are in a minimum.

$$\frac{d}{dq} \left[\frac{1}{a} \frac{da}{dq} \right] = -\frac{n_1}{q^2} - \frac{n_2}{(1+q)^2} - \frac{\varepsilon^2 n_3}{(1+\varepsilon q)^2}.$$

A simple implementation of this can be seen in the appendix.

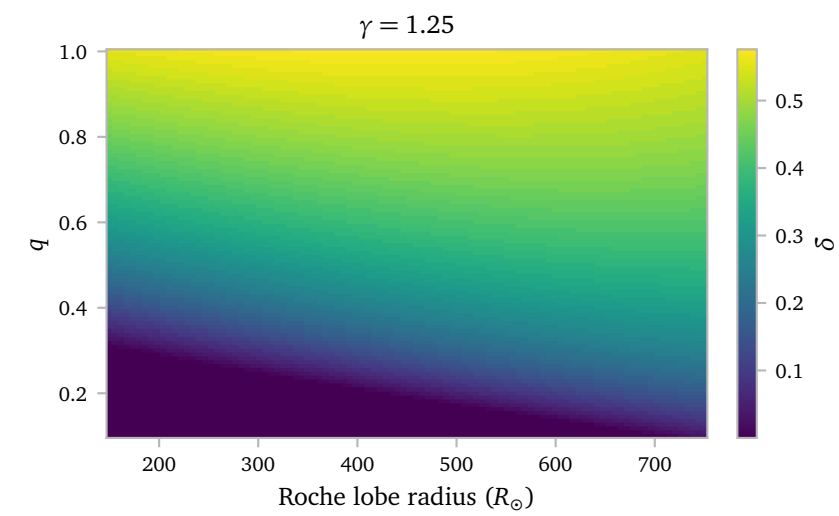
a.2 Results

As a first test of the model, I plot the fraction of the maximum star radius and the minimum separation as a function of the initial Roche lobe radius and mass ratio.

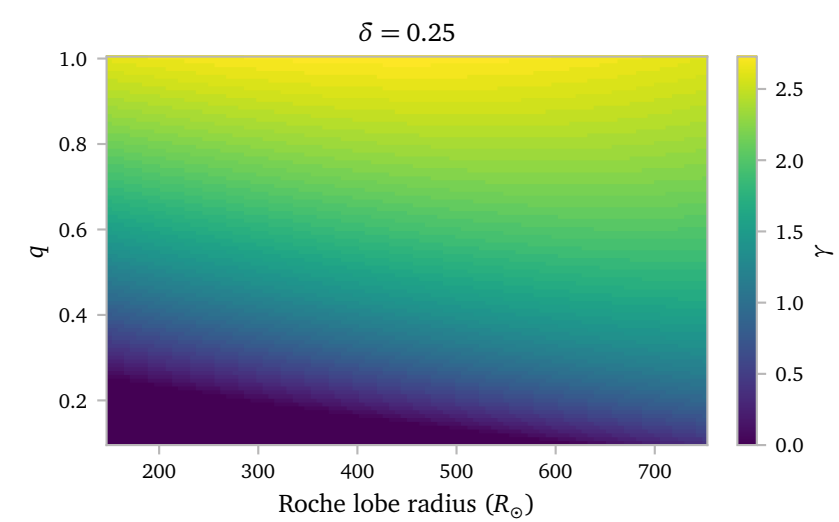


The ratio seems to mostly depend on the mass ratio, and not so much on the initial Roche lobe radius. I guess this makes sense, as the mass ratio significantly affects *how* the orbit responds to mass loss, while the initial separation does not.

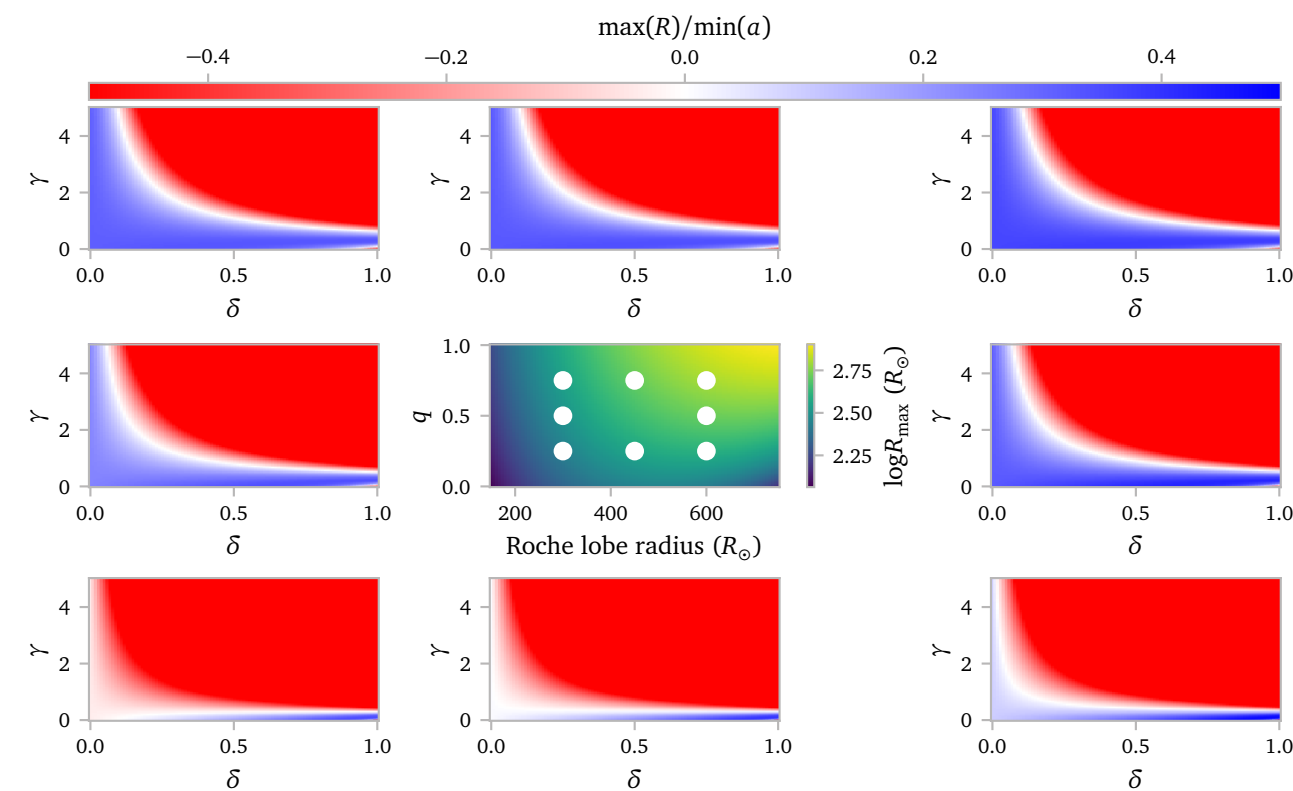
We can also look at the required δ for $\max R_{\text{star}} = \min a$.



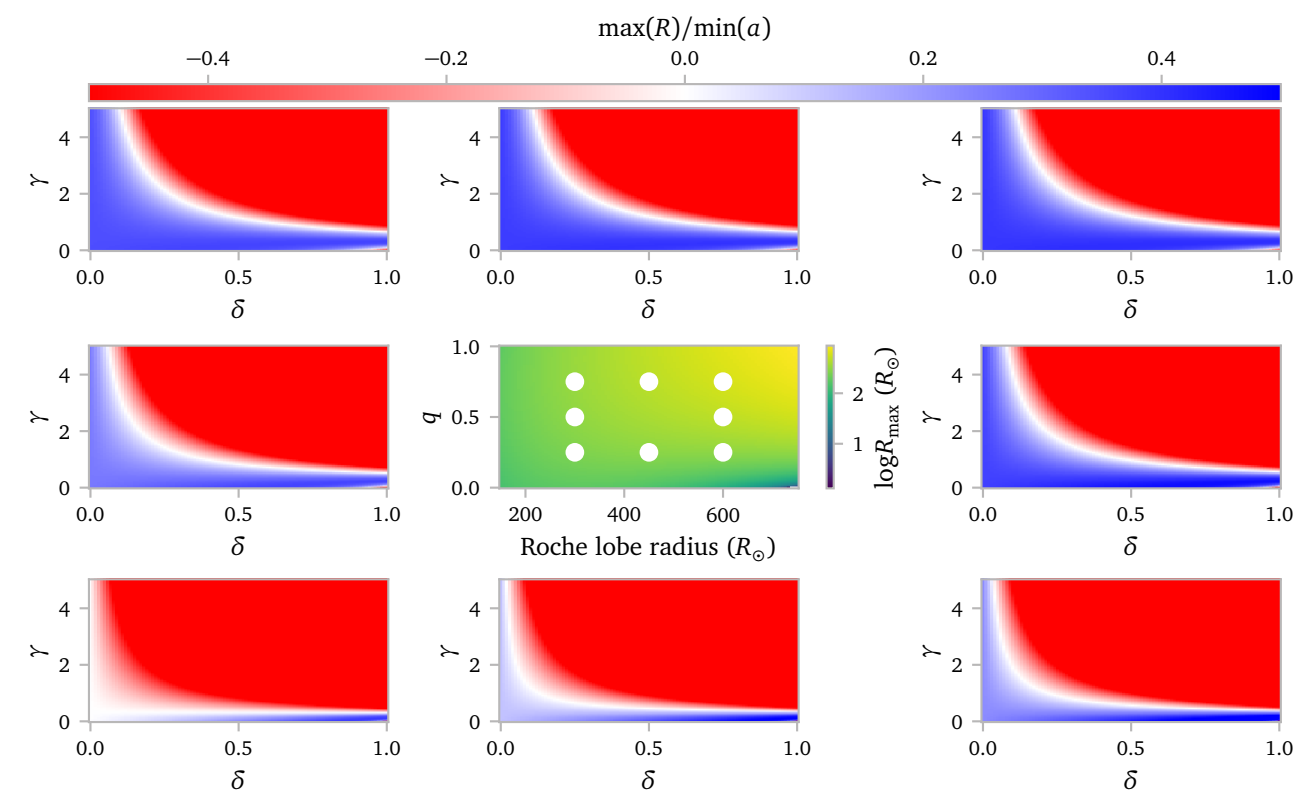
Obviously, the same can be done for γ :



But really, γ and δ are connected, so we would like to see them together. Showing this in a static plot is difficult, but I have tried:



As a test, we can also use the fit from the *MESA* models with stronger mass loss:



References

Soberman, G. E., Phinney, E. S., & van den Heuvel, E. P. J. (1997) *Stability criteria for mass transfer in binary stellar evolution..* [A&A327, 620.](#)

A Python Implementation

```
def a_over_a0(self, q=None):
    # table 5 soberman1997
    if q == None:
        q = self.qs
    term_1 = (q / self.q0) ** (2 * self.curA - 2)
    term_2 = ((1 + q) / (1 + self.q0)) ** (1 - 2 *
        ↪ self.curB)
    term_3 = ((1 + self.epsilon * q) / (1 +
        ↪ self.epsilon * self.q0)) ** (
        2 * self.curC + 3
    )
    return term_1 * term_2 * term_3

def da_over_a0_dq(self, q=None):
    if q == None:
        q = self.qs

    f = self.a_over_a0()

    n1 = 2 * self.curA - 2
    n2 = 1 - 2 * self.curB
    n3 = 2 * self.curC + 3

    dlogf = n1 / q + n2 / (1 + q) + n3 * self.epsilon
        ↪ / (1 + self.epsilon * q)

    return f * dlogf

def gprime(self, q):
    eps = self.epsilon
    n1 = 2 * self.curA - 2
    n2 = 1 - 2 * self.curB
    n3 = 2 * self.curC + 3

    gprime = -n1 / q**2 - n2 / (1 + q) ** 2 - n3 *
        ↪ eps**2 / (1 + eps * q) ** 2
    return gprime > 0

@property
def stationary_points(self):

    n1 = 2 * self.curA - 2
    n2 = 1 - 2 * self.curB
    n3 = 2 * self.curC + 3

    eps = self.epsilon

    a = eps * (n1 + n2 + n3)
    b = n1 * (1 + eps) + n2 + n3 * eps
    c = n1

    disc = b**2 - 4 * a * c

    if disc < 0:
        argmin = np.argmin(self.a_over_a0())
        return self.qs[argmin]

    sqrt_disc = np.sqrt(disc)

    q1 = (-b + sqrt_disc) / (2 * a)
    q2 = (-b - sqrt_disc) / (2 * a)
    roots = [q for q in [q1, q2] if q > 0]
    if len(roots) == 0:
        argmin = np.argmin(self.a_over_a0())
        return self.qs[argmin]
    roots = np.sort(roots)
    for root in roots:
        if self.gprime(root):
            argmin = np.argmin(self.a_over_a0())
            return np.max([root, self.qs[argmin]])

    argmin = np.argmin(self.a_over_a0())
    return self.qs[argmin]
```